SEND5232: Software Architecture and Design Project

Bereket Kiros – ugr/189518/16 – Section 2

PHASE 4 — Full-Stack Technology Selection and Justification

📚 Lihq'et Bookstore System

The Lihq'et Bookstore System is implemented using a carefully selected full-stack technology set designed to support scalability, maintainability, performance, and simplicity. The system follows a 3-tier layered architecture combined with MVC and Service Layer patterns, and each technology was chosen to align with these architectural decisions and the system's Architecturally Significant Requirements (ASRs).

Frontend (HTML, CSS, JavaScript):

The frontend is built using HTML, CSS, and vanilla JavaScript to provide a lightweight and responsive user interface for interacting with the bookstore system. This choice ensures simplicity, fast loading time, and ease of integration with backend APIs. JavaScript Fetch API is used to communicate with the backend asynchronously, supporting non-blocking operations and improving user experience. This technology stack aligns with usability and performance ASRs by ensuring quick rendering and smooth dynamic updates without page reloads.

Backend (Node.js + Express.js):

Node.js with Express.js is used for the backend to handle server-side logic and RESTful API communication. Node.js provides a non-blocking, event-driven architecture, making it highly efficient for handling multiple concurrent requests. Express.js enables structured routing and supports MVC architecture implementation, improving modularity and maintainability. The Service Layer is integrated to separate business logic from controllers, ensuring low coupling and high cohesion, which directly supports maintainability and scalability ASRs.

Database (MySQL):

MySQL is used as the relational database management system to store and manage book inventory data. It ensures data consistency, reliability, and ACID compliance. The relational structure is suitable for managing structured data such as books with attributes like title, author, price, and stock. SQL queries provide precise control over CRUD operations, supporting performance and data integrity ASRs. The database is accessed only through the backend layer, ensuring information hiding and security.

Architecture Style (MVC + Service Layer + 3-Tier Architecture):

The system follows a 3-tier architecture consisting of Presentation, Application, and Data layers. Within the backend, the MVC pattern is applied where Controllers handle HTTP requests, Services manage business logic, and Models handle database interactions. This separation improves maintainability, scalability, and testability. The Service Layer ensures reusable business logic and reduces redundancy, following the DRY principle.

Communication (REST API over HTTP):

The frontend and backend communicate using RESTful APIs over HTTP. This ensures loose coupling between layers and allows independent development and scalability of frontend and backend components. JSON is used as the data exchange format, providing lightweight and structured communication.

Justification Summary:

The selected technology stack was chosen to balance simplicity and scalability. It supports all key ASRs including performance (fast API responses), maintainability (MVC + Service Layer separation), usability (responsive frontend), and scalability (modular backend design). The combination of Node.js, Express, MySQL, and vanilla frontend technologies ensures a lightweight yet extensible system suitable for future enhancements such as authentication, search functionality, and online ordering features.